

Exercices : Récursivité

EXERCICE 1 - Puissance

En utilisant la définition du cours, écrivez une fonction **récursive** nommée `puissance` qui prend en paramètres deux entiers `x` et `n` et qui calcule x^n

💡 Votre fonction sera constituée d'un `if` qui traitera le cas de base ($n = 0$) et un `else` qui traitera les autres cas avec un **appel de la fonction à elle-même**. ⚠️ Chacun des cas doit avoir son `return`

EXERCICE 2 - Somme

Ci-dessous une fonction **itérative** (avec une boucle `for`) nommée `sommeI` qui prend en paramètre un entier `n` et calcule la somme des entiers 0 à `n` :

```
def sommeI(n):  
    s=0  
    for i in range(n+1) :  
        s= s+i  
    return s
```

1. Recopiez cette fonction et exécutez-la avec la valeur `100` .
2. Ecrivez maintenant une version **récursive** de la même fonction nommée `sommeR`

EXERCICE 3 - Plus Grand Commun Diviseur

En mathématiques le **Plus Grand Commun Diviseur** ou **PGCD** de deux nombres est le plus grand chiffre par lequel les deux nombres sont divisibles.

Écrivez une fonction **récursive** `pgcd(a,b)` renvoyant le plus grand diviseur commun de deux nombres `a` et `b` .

Vous testerez la fonction avec les valeurs `236` et `1292` , elle devra vous renvoyer `4`

💡 On utilisera le résultat mathématique suivant : $pgcd(a,b) = pgcd(b,r)$ où r est le reste de la division euclidienne de a par b . A vous de trouver le cas de base...

EXERCICE 4 - Tri

Écrivez une fonction **récursive** `is_sorted(li)` qui prend en paramètre une liste d'entiers `li` et décide si elle est trié par ordre croissant (en renvoyant `True`) ou pas (en renvoyant `False`)

EXERCICE 5 - La suite de Fibonacci

Comme abordé dans le cours, la célèbre suite de Fibonacci est définie de la manière suivante :

$$fibonacci(n) = \begin{cases} 1 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ fibonacci(n-1) + fibonacci(n-2) & \text{si } n > 1 \end{cases}$$

Vous trouverez ci-dessous une version itérative de la suite dans la fonction `fibonacciI` :

```
def fiboI(n):
    if n==0 :
        return 1
    elif n==1:
        return 1
    a = 1
    b = 1
    c = 0
    for i in range(1,n):
        c = a + b
        a = b
        b = c
    return c
```

1. Recopiez et exécutez la fonction avec la valeur 8
2. Ecrivez une version **récursive** de la suite dans une fonction nommée fiboR
3. Recopiez et exécutez le code suivant qui compare les deux versions de la suite :

```
from timeit import default_timer as timer
# Nombre d'éléments de la liste
nbelt=30
# Démarrage du chrono
debut_chrono = timer()
# Tri de la liste
for i in range (nbelt) :
    print (fiboI(i), end= " " )
# Fin du chrono et calcul de la durée
fin_chrono = timer()
duree = fin_chrono - debut_chrono
# Affichage du résultat
print("\n - Itératif : ",round(duree*1000),"millisecondes")
# Démarrage du chrono
debut_chrono = timer()
# Tri de la liste
for i in range (nbelt) :
    print (fiboR(i), end= " " )
# Fin du chrono et calcul de la durée
fin_chrono = timer()
duree = fin_chrono - debut_chrono
# Affichage du résultat
print("\n - Récursif : ",round(duree*1000),"millisecondes")
```

NOTE

Vous remarquerez que la version récursive de la suite est plus facile à comprendre et implémenter mais elle n'est pas forcément la plus performante... Nous réaborderons cet aspect dans notre cours sur la **programmation dynamique**.

EXERCICE 6 - Palindromes

On appelle **palindrome** un mot qui se lit dans les deux sens comme "selles", "lol", "121" ou "laval".

Pour la récursivité, on pourra dire que "**selles**" est un palindrome si : "**s**" = "**s**" et "**elle**" est un palindrome.

Pour la (les) condition (s) de sortie, à vous de trouver...

Ecrivez une fonction `estpalindrome(mot)` qui prend une chaîne de caractères et renvoie `True` si la chaîne est un palindrome ou `False` sinon.